Prepared by: S Arvind (Solutions Architect) Version: v1.1_202601

Summary

This document provides the details on how to deploy MaaS (metal as a service) for managing lifecycle of the baremetal nodes

Requirements

MaaS can be deployed in varieties of configurations as a virtual or physical instance based on the needs. Generally it is deployed as a Virtual machine in the environment. This section assume a single MaaS node providing the services, in a production setup the **minimum** recommendation are:

Resource	requirements
CPU	4-8 vCPUs
Memory	16GB
Storage	150GB disk (for caching OS images)
Network	1 or 10GBps NIC
OS	Ubuntu 22.04

OS version

In the current state all of the platform9 products are tested and certified to run on **ubuntu 22.04** LTS and hence it is recommended to user 22.04 LTS as the base OS version for deploying MaaS service in the environment.

Network requirements

The MaaS host needs access to below network for a successful deployment and management experience

- External Internet: To download updates and sync the deployment images.
- IPMI network: To connect with the Baremetal nodes and perform boot/reboot operation over IPMI connection.
- Dedicated PXE network/VLAN segment: To distribute TFTP service, provide IP to the Baremetal using the DHCP service in the MaaS host and providing the images that will be used to re-image the baremetal node.

For further reading about the requirements: https://maas.io/docs/installation-requirements

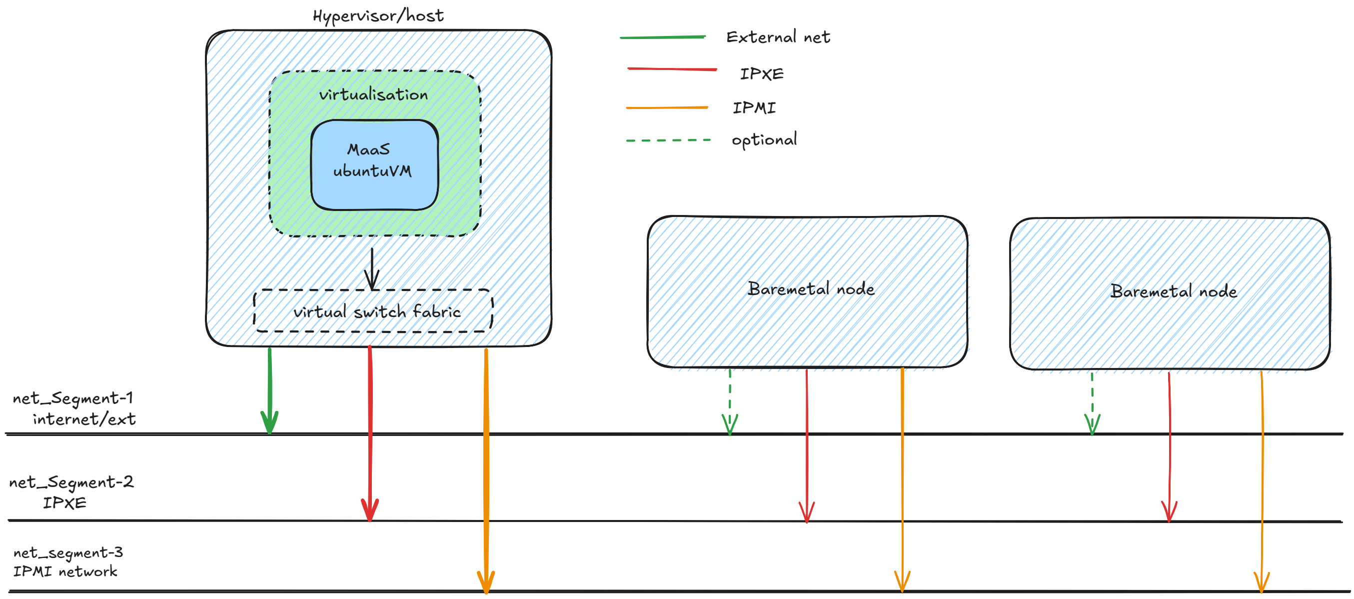
⚠ Network VLANs

MaaS manual and automation deployment have been tested with Native VLAN tagging functionality only in the switching infrastructure. Hence it is recommended to configure the interconnecting network using Native VLAN tags only.

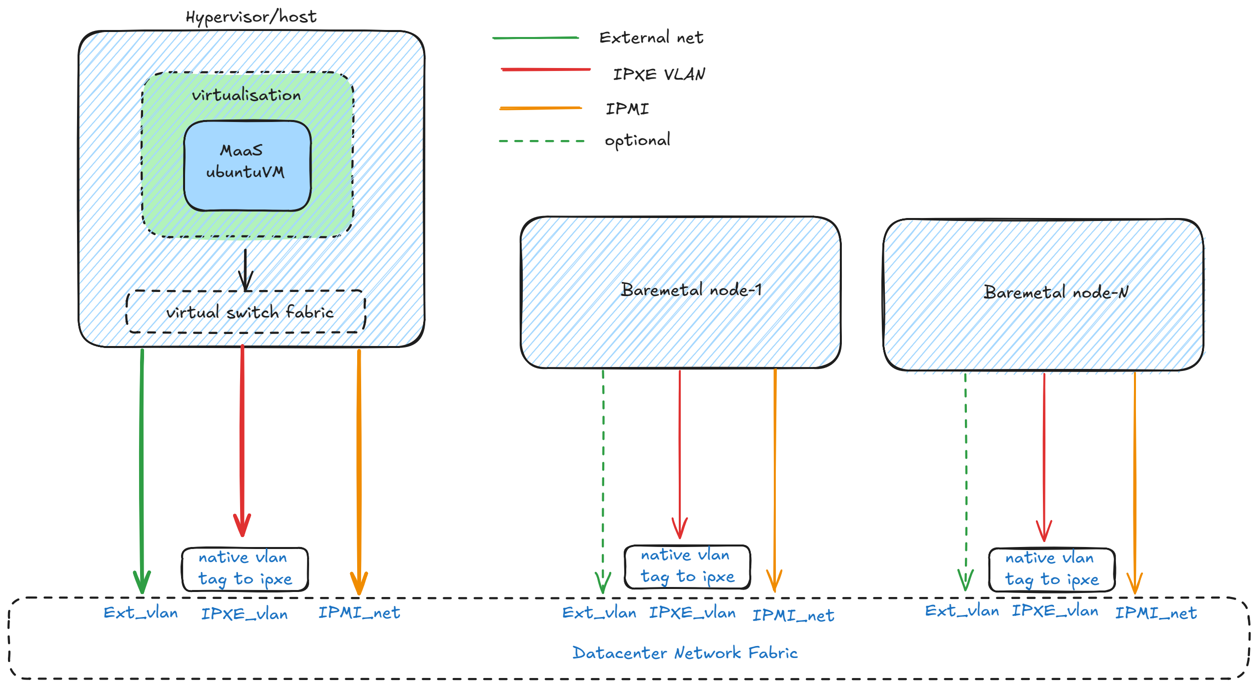
Network Architecture for MaaS deployment

MaaS service can be deployed as a VM instance or on a baremetal node depending on the requirements. The general network connectivity/architecture needed for MaaS functionality when deployed as virtual instance should look like one of the below:

General network architecture



Architecture with native-vlan configuration



Maas service sync the required baremetal images from the upstream repos and can be configured either be as DHCP server or can use existing DHCP relays setup to distribute the IPs for the baremetal during re-imaging operations. As a best practice, it is recommended to make use of MaaS provided DHCP server which will distribute the IP to the target hosts during installation process. The service will also need the IPMI network of the baremetal nodes to be accessible to trigger the power on/off/reboot operations during the setup.

The Baremetal nodes will need IPMI and the connection to IPXE network segment in which the MaaS server will provide the IP and the images to boot them. Internet/external network access/connections can be optional since the MaaS server hosts an internal proxy that provides the required access to internet to the baremetal during installation.

PXE boot network configurations

In VLAN segmented network with trunked interconnects between MaaS and the baremetal nodes, it is generally recommended to have the IPXE network vlan used for Maas, to be mapped as ***native vlan*** tag on the interfaces connecting the baremetal nodes as the PXE firmware, depending in the hardware vendor, is generally not capable to handle VLAN tags

---

Installation and setup

Preparing the host

- Deploy a fresh ubuntu 22.04 LTS host and configure the login user with password less sudo access. As a root user, create a sudoer file and set the following option for the desired user:

```
$ vim /etc/sudoers.d/90-cloud-init-users
<username> ALL=(ALL) NOPASSWD:ALL
```

- As the sudo user run the update

```
sudo apt-get update
sudo apt-get upgrade
```

- Generate a fresh ssh keys which will be used in the later stages for ssh access to the baremetal nodes. Run the command and press `enter` to the inputs to set all the options as default:

```
ssh-keygen
```

Install and configure Maas service (manual steps)

- Install MaaS packages:

```
sudo apt-add-repository ppa:maas/3.5 -y
sudo apt update
sudo apt-get -y install maas
```

- Disable and remove the `systemd-timesyncd` service:

```
sudo systemctl disable --now systemd-timesyncd
sudo apt-get purge systemd-timesyncd -y
```

- Install and configure `postgresql` service:

```
sudo apt install -y postgresql
sudo -i -u postgres psql -c "CREATE USER \"$DBUSER\" WITH ENCRYPTED PASSWORD
'$DBPASS'"
sudo -i -u postgres createdb -O "$DBUSER" "$DBNAME"
```

- Edit `/etc/postgresql/14/main/pg_hba.conf` by adding a line for the newly created:

```
 host    $MAAS_DBNAME    $MAAS_DBUSER    0/0     md5
```

- Create Maas admin user

```
sudo maas createadmin --username "$MAAS_USERNAME" --password
"$MAAS_PASSWORD" --email "$MAAS_EMAIL"
```

The maas service will be ready and will be listening on PORT 5240 on the host and the UI will be accessible over http://{ip-addr}:5240/

Install and configure Maas service (Automated steps)

- Perform the "prepare host steps"
- Create a file with name maas-configure.sh on the host with content as below:

```
#!/bin/bash

# check for packages and install if not present
check_packages() {
    # Check if required packages are installed
    sudo apt-get update
    for package in "$@"; do
        if ! dpkg -l | grep -q "^ii  $package "; then
            echo "Package $package is not installed. Installing..."
            sudo apt install -y "$package"
        else
            echo "Package $package is already installed."
        fi
    done
}

# check network connectivity and select the best interface with default
gateway based on metric
check_network_connectivity() {
    # Get all default gateway interfaces with their metrics
    INTERFACES=($(ip -4 route show default | awk '{print $5, $NF}' | sort -
k2n | awk '{print $1}'))
    if [[ ${#INTERFACES[@]} -eq 0 ]]; then
        echo "No default gateway interfaces found."
        return 1  # False
    fi
    echo "Available default gateway interfaces: ${INTERFACES[*]}"

    # Pick the interface with the lowest metric
    CHOSEN_IFACE=${INTERFACES[0]}
    if [[ ${#INTERFACES[@]} -gt 1 ]]; then
        echo "Multiple default gateway interfaces found. Testing
connectivity..."

        for IFACE in "${INTERFACES[@]}"; do
            echo "Testing connectivity via $IFACE..."
```

```
            if ping -I "$IFACE" -c 3 -W 2 8.8.8.8 &>/dev/null; then
                echo "Selected interface: $IFACE (successful ping)"
                CHOSEN_IFACE=$IFACE
                break
            fi
        done
    fi
    echo "selected interface: $CHOSEN_IFACE"

    # Validate internet connectivity
    if ! ping -I "$CHOSEN_IFACE" -c 3 -W 2 8.8.8.8 &>/dev/null; then
        echo "Internet connectivity test failed: Cannot reach 8.8.8.8 via
$CHOSEN_IFACE."
        return 1  # False
    fi

    # Validate DNS resolution
    if ! nslookup google.com &>/dev/null; then
        echo "DNS resolution test failed: Cannot resolve google.com."
        return 1  # False
    fi

    echo "All network checks passed using interface: $CHOSEN_IFACE"
    return 0  # True
}

install_maas() {
    echo
    sudo apt update
    sudo systemctl disable --now systemd-timesyncd
    echo "Disabling systemd-timesyncd... to avoid conflicts with chrony
services"
    sudo apt-get remote systemd-timesyncd -y
       # Check if PostgreSQL is installed
    if dpkg -l | grep -q "^ii  postgresql "; then
      echo "PostgreSQL is already installed. Skipping database user and
creation steps."
    else
      check_packages "postgresql"
      sudo -i -u postgres psql -c "CREATE USER \"$DB_USER\" WITH ENCRYPTED
PASSWORD '$DB_PASSWORD'"
      sudo -i -u postgres createdb -O "$DB_USER" "$DB_NAME"
      sudo apt-add-repository ppa:maas/3.5 -y
      sudo apt update
      sudo apt-get -y install maas
      echo "host    $DB_NAME    $DB_USER    0/0    md5" | sudo tee -a
```

```
/etc/postgresql/14/main/pg_hba.conf > /dev/null
      sudo maas createadmin --username "$MAAS_USERNAME" --password
"$MAAS_PASSWORD" --email "$MAAS_EMAIL"
      echo "MAAS installation and configuration complete."
    fi
}

# main script execution
echo "Starting MAAS installation script..."
echo " "
echo "Checking network connectivity..."
if check_network_connectivity; then
    echo "Network is reachable."
    echo " "
    echo "Checking and installing additional required packages..."
    check_packages "tree" "net-tools" "ansible" "python3-openstackclient"
    echo " "
    read -p "Enter MAAS username: " MAAS_USERNAME
    read -s -p "Enter MAAS password: " MAAS_PASSWORD
    echo
    read -p "Enter MAAS email: " MAAS_EMAIL
    read -p "Enter PostgreSQL database name: " DB_NAME
    read -p "Enter PostgreSQL username: " DB_USER
    read -s -p "Enter PostgreSQL password: " DB_PASSWORD
    echo
    echo "Proceeding with MAAS installation..."
    install_maas
else
    echo "Network check failed."
    echo "Please check your network configuration and try again."
fi
```

- Set execute permission:

```
sudo chmod a+x maas-configure.sh
```

- Execute the script and provide the requested details:

```
./maas-configure.sh
Starting MAAS installation script...

Checking network connectivity...
...
```

```
Enter MAAS username:
Enter MAAS password:
Enter MAAS email:
Enter PostgreSQL database name:
Enter PostgreSQL username:
Enter PostgreSQL password:
```

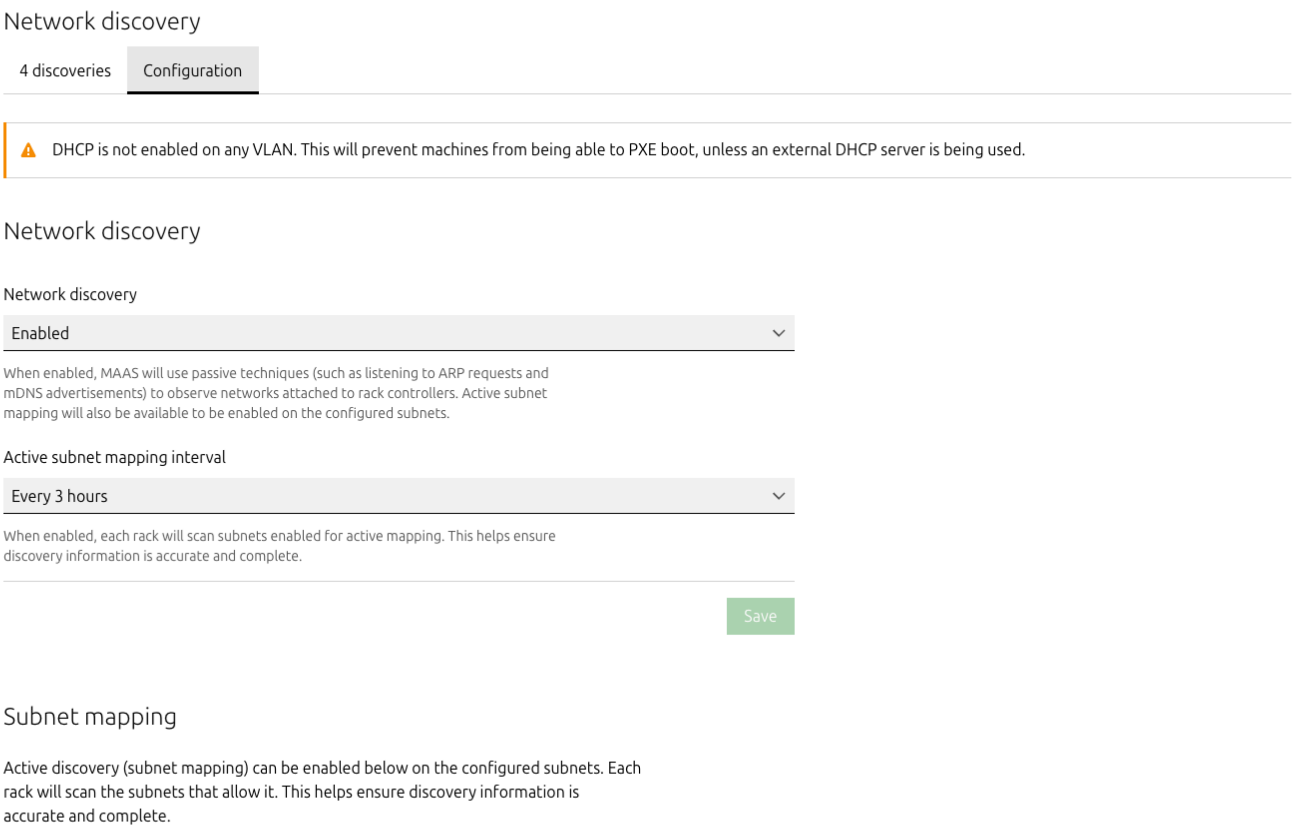
- Post the execution the MaaS service should be configured and UI will be accessible over port 5240

> **Note**
>
> - The script is to be executed on a fresh ubuntu host and will exit if it finds the MaaS is pre-installed.
> - It is not recommended to re-run the script on an already running MaaS setup.

Managing Hosts with MaaS UI

First time login

- At the first login the UI will present with some options that can be configured or skipped as per the needs. One of the first page:



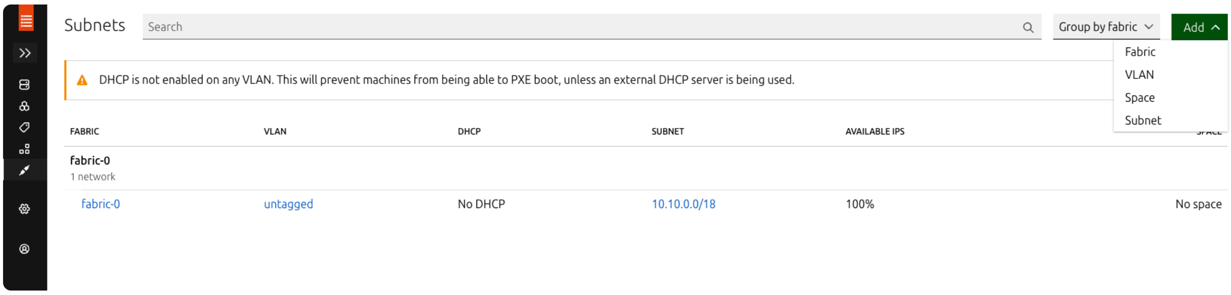
IMP: Depending on the needs **Network discovery** option can be enabled or disabled. With this set to `enabled`, MAAS passively monitors ARP traffic to detect devices on the network. It records IP and MAC addresses, tracking changes over time. This aid the hardware auto-discovery feature where if a baremetal host is available in the PXE network Maas will perform the commissioning/introspection operation and will add it in its inventory of hosts that can be deployed/re-imaged. In general to have better control of the operation **it is recommended to keep this option `disabled` in general

reference: https://maas.io/docs/about-networking

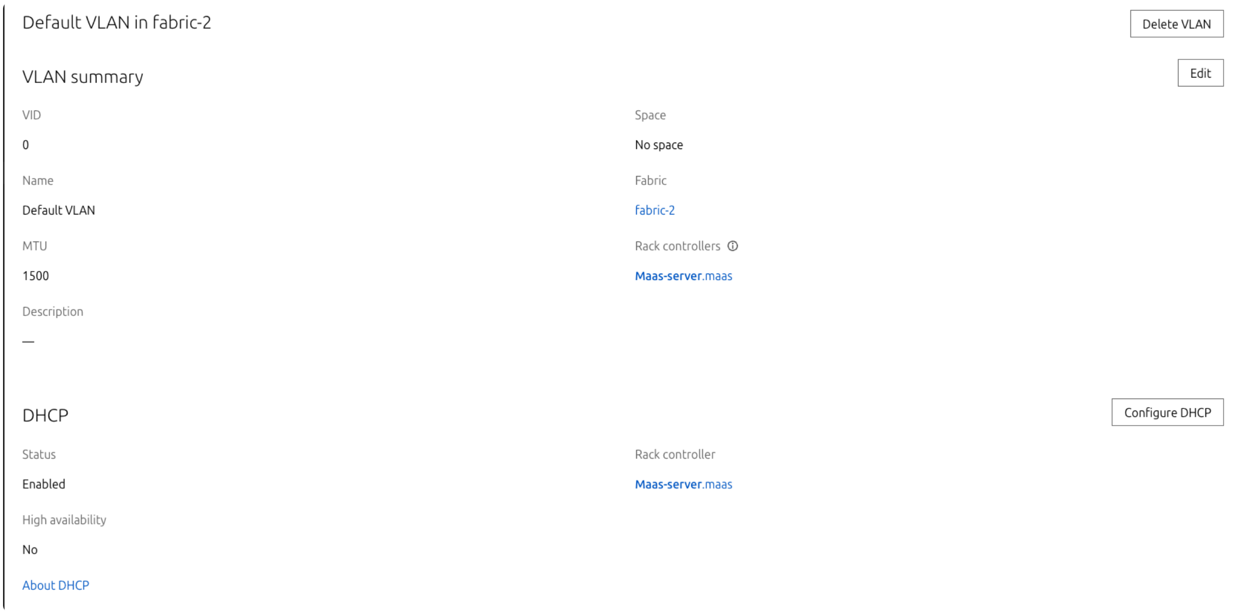
Setup DHCP

Initially the Maas service will auto discover networks which will be listed as Fabric-0, frabri-1 etc. Under each fabric the VLAN or the untagged subnets can be added.

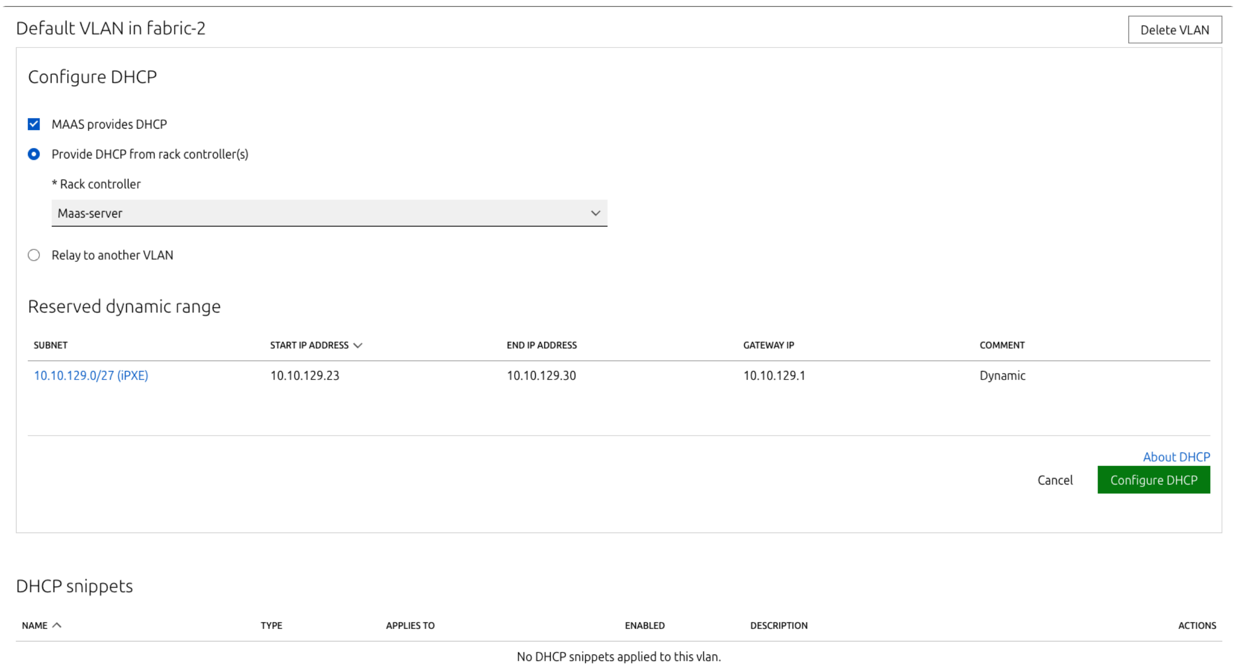
- To add the PXE subnet with DHCP to be used for Baremetal boot up:
  - Navigate to **Subnets** > VLAN (select your VLAN ID) or untagged > Configure DHCP.



- Select the appropriate DHCP options.



- Save and apply settings.



Dynamic range reservation field:

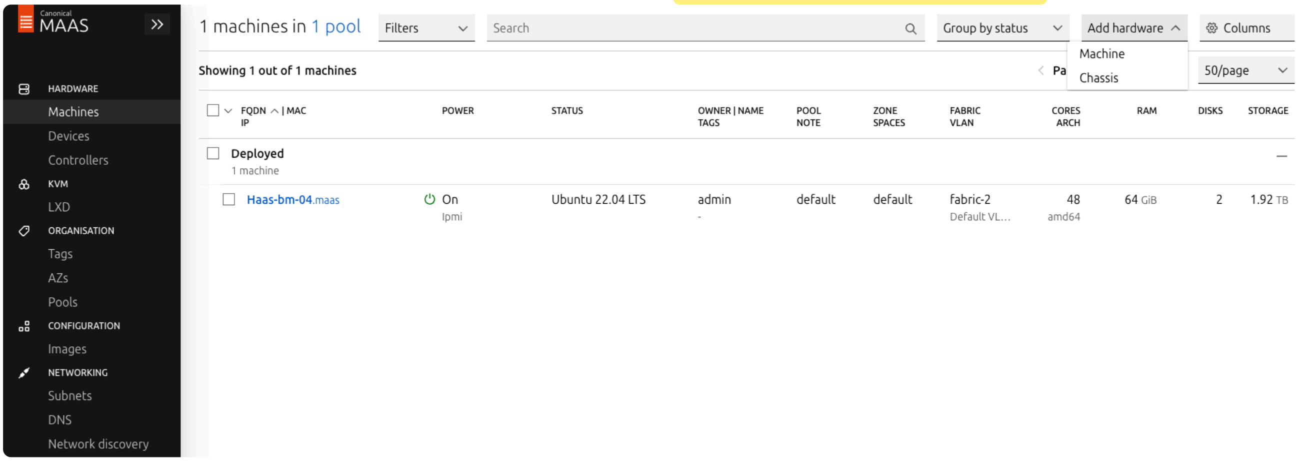
This field in the DHCP configuration settings screen enforces the ip reservation during dhcp configuration. **The reservation basically exclude distributing any IPs the defined set of IP range in it.** This field can then be modified again to remove any reservation if not necessary for a given environment.

Commission hosts

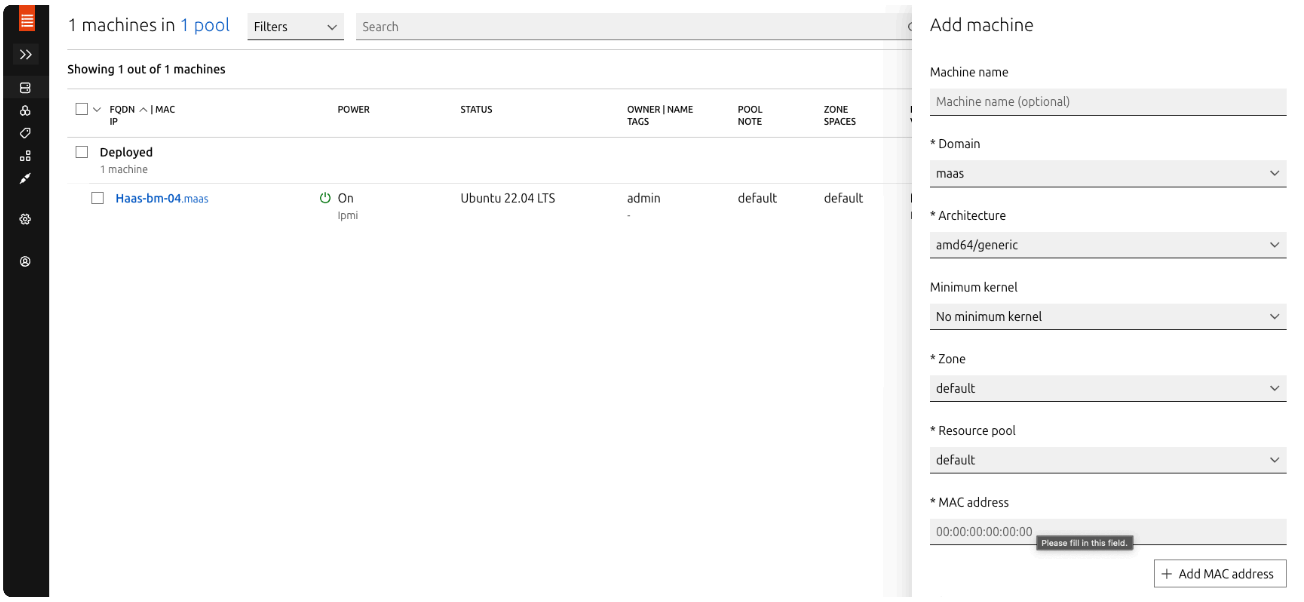
⚠ PXE configuration on the Baremetal Nodes

Before adding the target baremetal nodes to the MaaS, Ensure the network boot services are enabled on the respective NIC which will be part of the MaaS iPXE network segment. Ensure the Baremetal nodes are able to network boot and acquire the PXE IP provided by MaaS on the designated NIC.

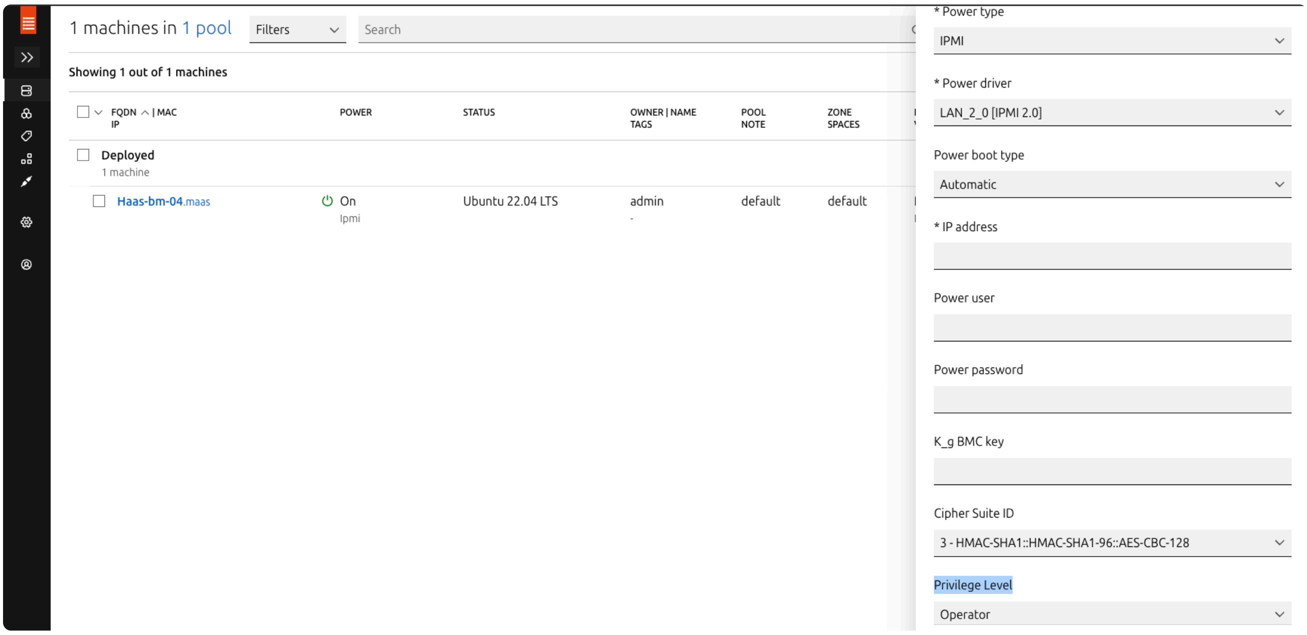
- From the Machines section goto add hardware > machines:



- Enter machine details:

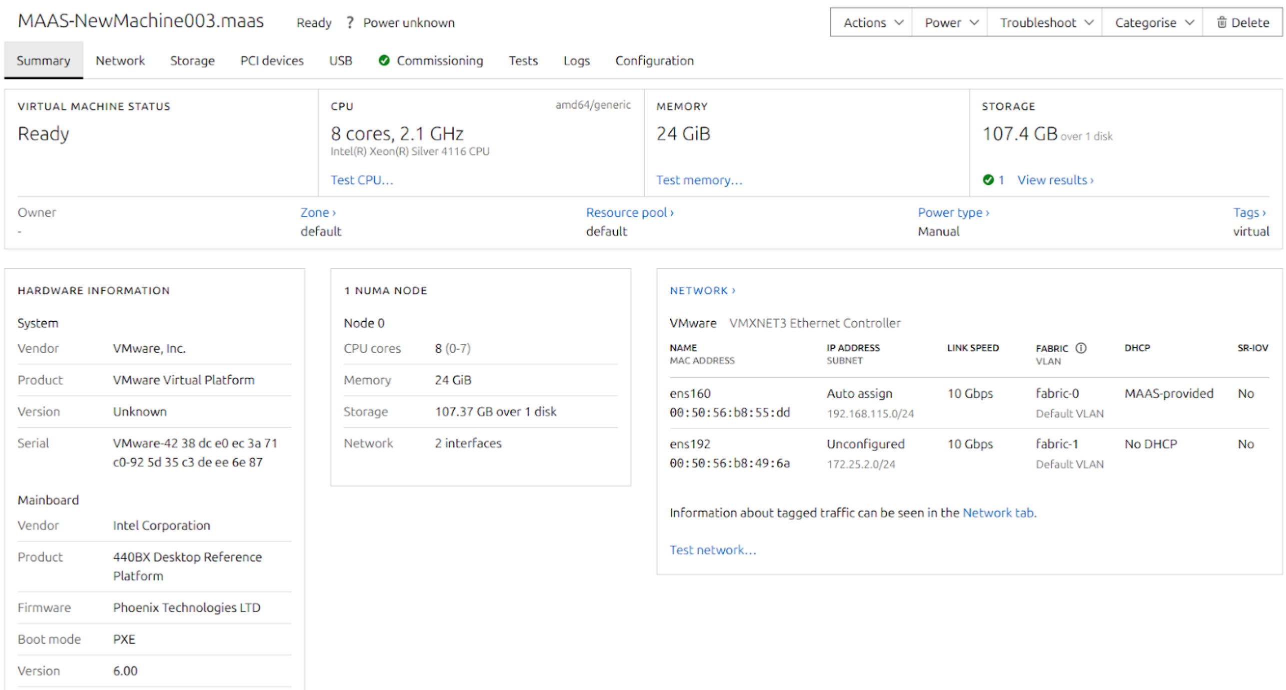


  - set the **Minimum kernel field** to jammy(ga-22-04)
- Select the Power type and set IPMI as the power option. Provide the necessary IPMI IP, username, password:



- Set the privilege level to Operator and save the settings.

- This process will trigger **The commissioning phase and collect all info regarding the machine** and then will power it down.
- Open the BMC (ILO/DRAC) console of the respective baremetal node.
- Post commissioning the Machine will be listed in the Ready state and the machine profile will be populated with the information and capabilities after the successful introspection.



Deploy hosts

tip

For the Baremetal to be deployed, it should be in **READY** state

- For the Machines listed as ready, click the actions dropdown and the deploy button:

- In the deploy section fill the set the release to ubuntu 22.04 and below options:

Deploy

OS

Ubuntu

Release

Ubuntu 22.04 LTS "Jammy Jellyfish"

Kernel

No minimum kernel

Deployment target

- (x) Deploy to disk
- ( ) Deploy in memory

  No disk layout will be applied during deployment. All system data will be reset upon reboot or shutdown.

Customise options

- [ ] Register as MAAS KVM host. KVM docs
- [ ] Cloud-init user-data... Cloud-init docs
- [ ] Periodically sync hardware ⓘ Hardware sync docs

  Hardware sync interval: 15 minutes - Admins can change this in the global settings.

 Cancel | Deploy machine

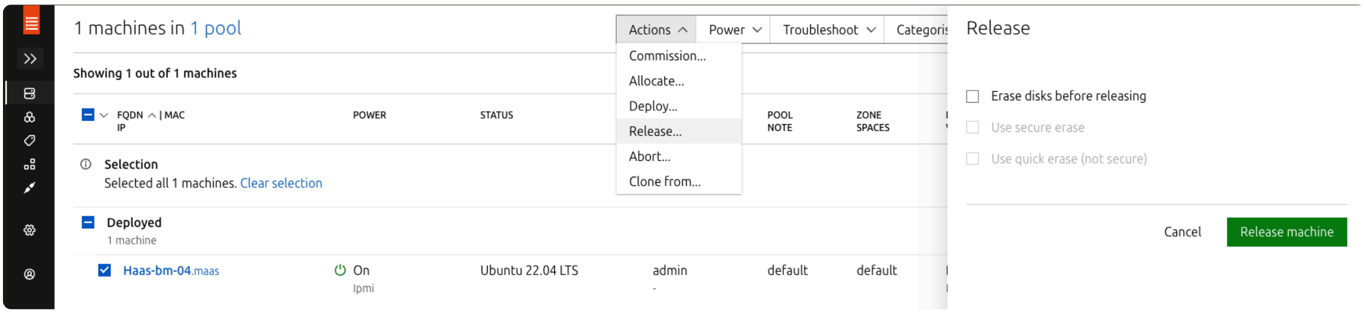
Customised host configuration with cloud-init

- Copy the sample file found at: https://github.com/Platform9-Community/sa-automation/blob/main/Maas_installer_script/cloud-init-samples/01-bond_with_vlan-example.yaml locally.
- Modify it as per requirement.
- Select the cloud-init user-data flag and copy the modified yaml file in the section and click the `deploy machine` option

Post the deployment, the machine will change state to `Deployed` in the UI and it should be reachable over the SSH with the networking and other configuration done with the cloud-init.

Releasing host from MaaS

To perform the Deploy action or remove the host from MaaS control the release action should be used. To release a host, click the action button after selecting the host and select "release" from the drop down:



- With "**erase disk before releasing**" selected, the host disk will be wiped and the power down at the end of the process.

Workflow of MaaS operations